

ASMIT SAHU

23052231

Objective: Introduce deep learning basics with neural networks.

Modify the model by changing: Hidden layer neurons from 128 $\rightarrow$ 64, Dropout rate from 0.2 $\rightarrow$ 0.4, Train the model for exactly 5 epochs.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(10, activation='softmax')
])

print(model.summary())
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
 11490434/11490434 ————— 0s 0us/step
 Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	50,240
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 50,890 (198.79 KB)
 Trainable params: 50,890 (198.79 KB)
 Non-trainable params: 0 (0.00 B)
 None

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                   epochs=5,
                   batch_size=32,
                   validation_data=(x_test, y_test))

test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"\n✅ Test Accuracy: {test_accuracy * 100:.2f}%")
```

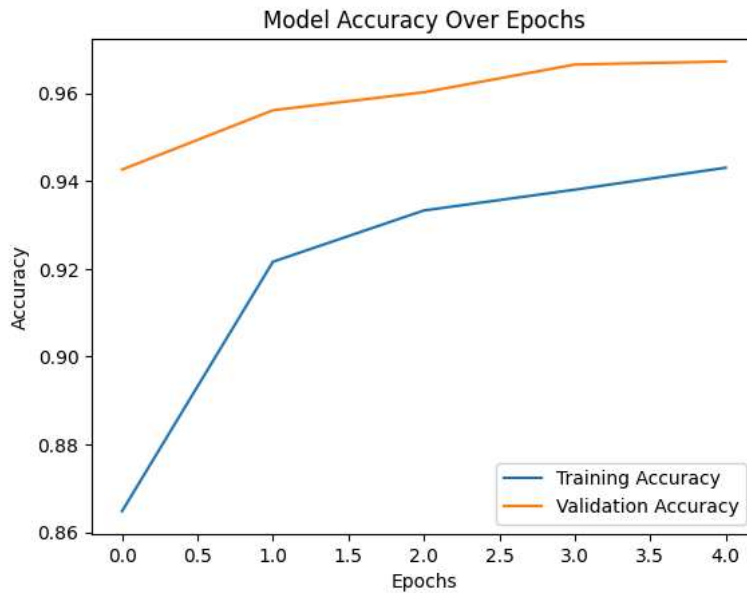
```
Epoch 1/5
1875/1875 ————— 9s 4ms/step - accuracy: 0.7823 - loss: 0.7089 - val_accuracy: 0.9426 - val_loss: 0.2008
Epoch 2/5
1875/1875 ————— 6s 3ms/step - accuracy: 0.9188 - loss: 0.2750 - val_accuracy: 0.9561 - val_loss: 0.1467
Epoch 3/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9321 - loss: 0.2311 - val_accuracy: 0.9602 - val_loss: 0.1281
Epoch 4/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9365 - loss: 0.2084 - val_accuracy: 0.9665 - val_loss: 0.1146
```

Epoch 5/5

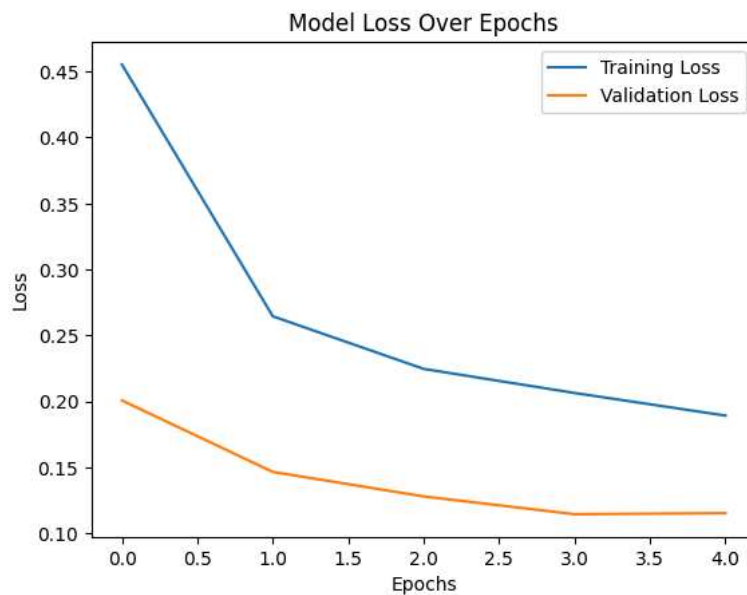
1875/1875 — 5s 3ms/step - accuracy: 0.9426 - loss: 0.1879 - val_accuracy: 0.9672 - val_loss: 0.1154
313/313 — 1s 3ms/step - accuracy: 0.9613 - loss: 0.1361

✓ Test Accuracy: 96.72%

```
plt.figure()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Model Accuracy Over Epochs')
plt.legend()
plt.show()
```



```
plt.figure()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Model Loss Over Epochs')
plt.legend()
plt.show()
```



Explain why the accuracy changed compared to the original model. (Mention this in your lab report too ?

Reducing the hidden neurons from 128 to 64 decreased the model's learning capacity, so it could capture fewer complex patterns. Increasing the dropout from 0.2 to 0.4 caused more neurons to be randomly deactivated during training, which improves regularization but lowers training accuracy. As a result, the model shows slightly lower training accuracy but better generalization, reducing the risk of overfitting.